

EXPERIMENT 5

DATA VISUALIZATION USING MATPLOTLIB

Aim:

To create and analyze various data visualizations such as Line Charts, Bar Charts, Pie Charts, Histograms, Scatter Plots, and Box Plots using Matplotlib on a healthcare stroke dataset for effective data interpretation.

Algorithm 1 — Line chart

Step 1: Import Matplotlib and Pandas.

Step 2: Load the healthcare stroke dataset.

Step 3: Divide patients into age groups.

Step 4: Calculate average glucose level for each age group.

Step 5: Use `plot()` function to create the line chart.

Step 6: Add title and axis labels.

Step 7: Display the chart.

Program:

```
import pandas as pd

import matplotlib.pyplot as plt

df = pd.read_csv("healthcare-dataset-stroke-data(3).csv")

age_groups = pd.cut(

    df["age"],

    bins=[0, 20, 40, 60, 80, 100],

    labels=["0-20", "21-40", "41-60", "61-80", "81-100"]

)

avg_glucose = df.groupby(

    age_groups,

    observed=False

)["avg_glucose_level"].mean()

plt.plot(avg_glucose.index, avg_glucose.values, marker='o')

plt.title("Average Glucose Level by Age Group")
```

```
plt.xlabel("Age Group")

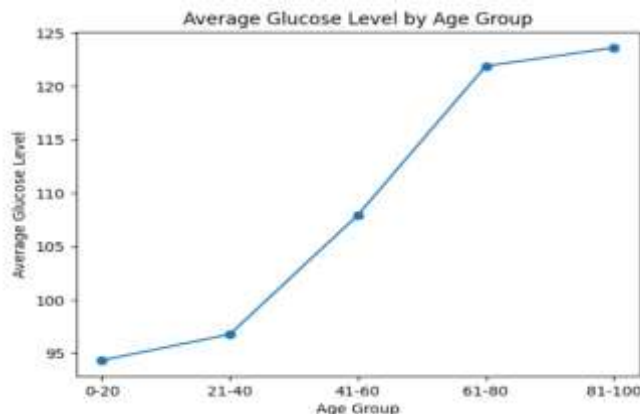
plt.ylabel("Average Glucose Level")

plt.show()
```

Output:

Average Glucose Level by Age Group

The line chart shows the variation in average glucose level across different age groups.



Algorithm 2 — Bar chart

- Step 1: Select patients who experienced stroke.
- Step 2: Group stroke cases according to gender.
- Step 3: Count the number of stroke cases for each gender.
- Step 4: Use `bar()` function to create the bar chart.
- Step 5: Add chart title and labels.
- Step 6: Display the chart.

Program:

```
stroke_gender = df[df["stroke"] == 1]["gender"].value_counts()

print(stroke_gender)

plt.bar(stroke_gender.index, stroke_gender.values)

plt.title("Stroke Cases by Gender")

plt.xlabel("Gender")

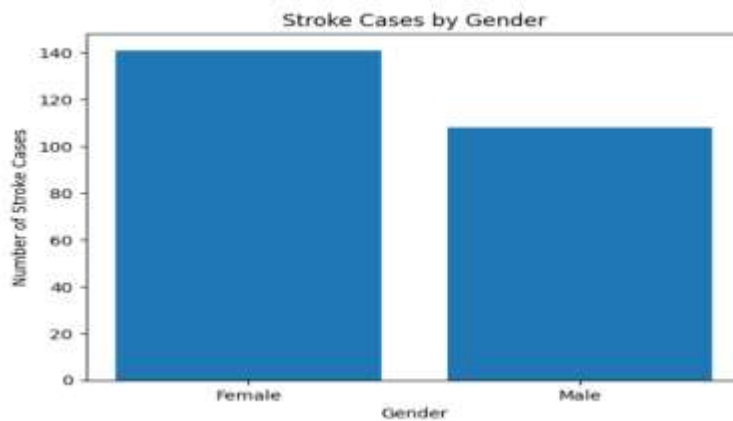
plt.ylabel("Number of Stroke Cases")

plt.show()
```

Output:

Stroke Cases by Gender

The bar chart compares the number of stroke cases among different genders.



Algorithm 3 — Pie chart

- Step 1: Count patients with and without stroke.
- Step 2: Create labels for the categories.
- Step 3: Apply `pie()` function.
- Step 4: Add percentage labels using `autopct`.
- Step 5: Add chart title.
- Step 6: Display the chart.

Program:

```
stroke_counts = df["stroke"].value_counts()
```

```
labels = ["No Stroke", "Stroke"]
```

```
plt.pie(
```

```
    stroke_counts,
```

```
    labels=labels,
```

```
    autopct='%1.1f%%'
```

```
)
```

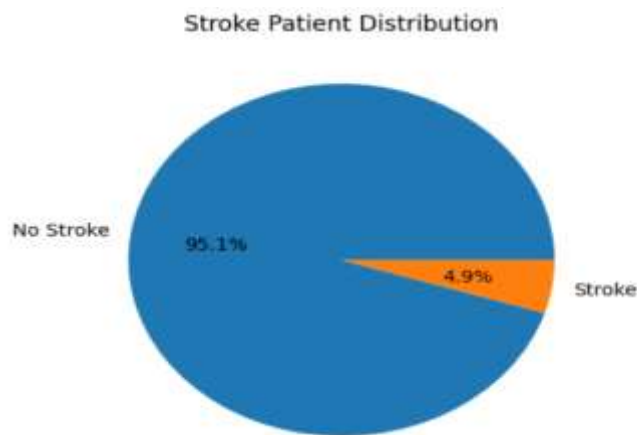
```
plt.title("Stroke Patient Distribution")
```

```
plt.show()
```

Output:

Stroke Patient Distribution

The pie chart shows the percentage distribution of patients with and without stroke.



Algorithm 4 — Histogram

- Step 1: Select the `age` column from the dataset.
- Step 2: Use `hist()` function.
- Step 3: Divide the data into suitable intervals.
- Step 4: Add title and axis labels.
- Step 5: Display the histogram.

Program:

```
plt.hist(df["age"], bins=10)
```

```
plt.title("Age Distribution of Patients")
```

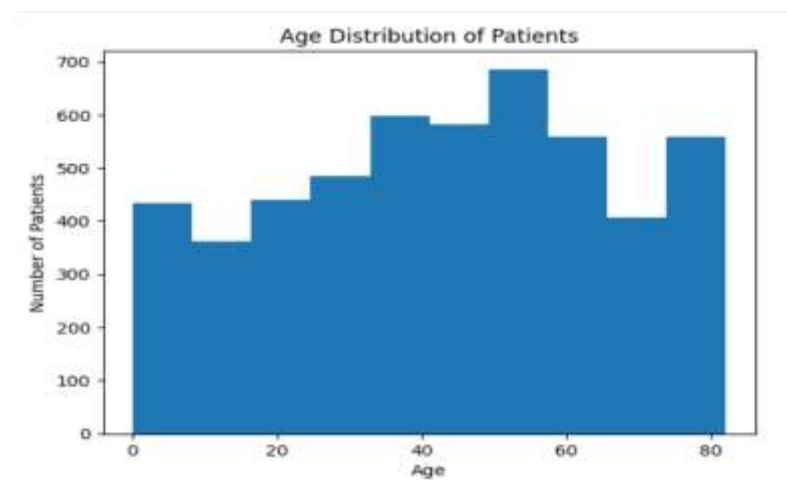
```
plt.xlabel("Age")
```

```
plt.ylabel("Number of Patients")
```

```
plt.show()
```

Output:

Age Distribution of Patients



Algorithm 5 — Scatter plot

- Step 1: Select `age` and `avg_glucose_level` columns.
- Step 2: Use `scatter()` function.
- Step 3: Plot age on the X-axis.
- Step 4: Plot average glucose level on the Y-axis.
- Step 5: Add title and axis labels.
- Step 6: Display the scatter plot.

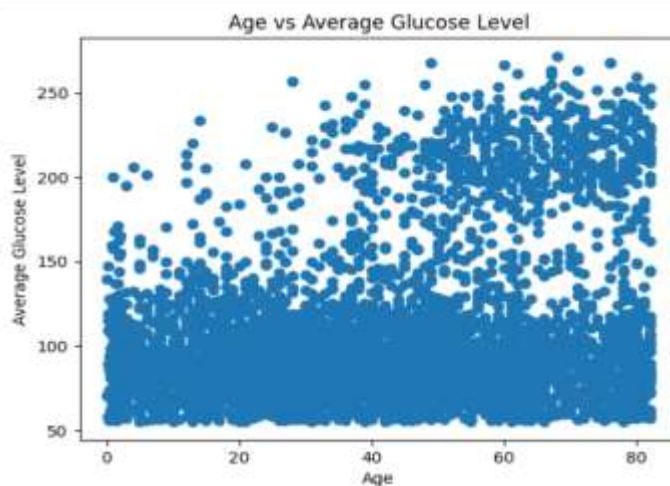
Program:

```
plt.scatter(  
    df["age"],  
    df["avg_glucose_level"]  
)  
  
plt.title("Age vs Average Glucose Level")  
  
plt.xlabel("Age")  
  
plt.ylabel("Average Glucose Level")  
  
  
plt.show()
```

Output:

Age vs Average Glucose Level

The scatter plot shows the relationship between patient age and average glucose level.



Algorithm 6 — Box plot

- Step 1: Select the `bmi` column.
- Step 2: Remove missing BMI values.

Step 3: Use `boxplot()` function.

Step 4: Add title and axis labels.

Step 5: Display the box plot.

Step 6: Identify possible outliers.

Program:

```
bmi_data = df["bmi"].dropna()
```

```
plt.boxplot(bmi_data)
```

```
plt.title("BMI Distribution")
```

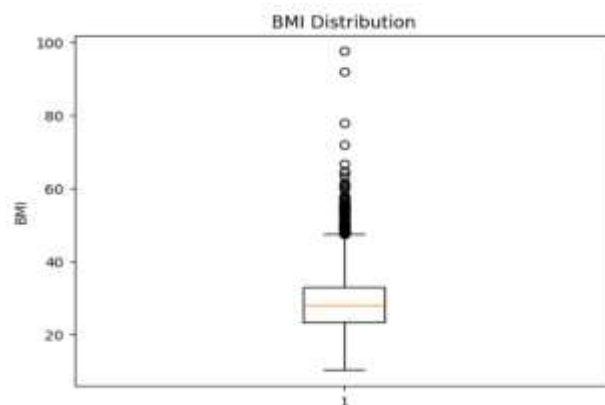
```
plt.ylabel("BMI")
```

```
plt.show()
```

Output:

BMI Distribution

The box plot represents the distribution of BMI values and helps identify potential outliers in the healthcare dataset.



Result:

Thus, various visualization techniques such as Line Charts, Bar Charts, Pie Charts, Histograms, Scatter Plots, and Box Plots were successfully created using Matplotlib on the healthcare stroke dataset. The graphical representations helped in understanding patient distributions, stroke patterns, glucose levels, age distribution, and BMI variations.

